Algorithmic Implementations and Evaluation for the Job Shop Scheduling Problem

A.H.M Towfique Mahmud (2005003), Md Zim Mim Siddiqee Sowdha (2005004), Kowshik Saha Kabya (2005006), Md Rafiul Islam Nijamy (2005035), Ha Meem (2005055), and Sadnam Faiyaz (2005111)

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)
Course: CSE 462 - Algorithm Engineering Sessional

April 10, 2026

Abstract

The Job Shop Scheduling Problem (JSSP) is a notoriously difficult combinatorial optimization problem with profound implications in manufacturing, cloud computing, and logistics. This report presents a comprehensive study of three distinct algorithmic approaches to solving the JSSP: an exact Branch and Bound (B&B) algorithm, a greedy heuristic known as Most Work Remaining (MWKR), and a metaheuristic Simulated Annealing (SA) approach. We evaluate these algorithms on 1,000 randomly sampled instances from the modern Starjob dataset. Our baseline findings reveal a strict trade-off between computational speed and schedule optimality. MWKR provides near-instantaneous schedules but suffers from a high optimality gap (averaging 70.65% accuracy), whereas baseline SA achieves an accuracy of 88.20% at a higher computational cost. We further present an improved SA variant incorporating the Nowicki–Smutnicki block-boundary neighbourhood across all critical paths, a reheat-and-restart mechanism, and steepest-descent hill climbing, which raises the average accuracy to **90.03%** and outperforms the baseline on 849 out of 1,000 instances. Additionally, an enhanced MWKR+SPT heuristic is introduced to break ties and minimize machine idle time.

Keywords— Job Shop Scheduling, Combinatorial Optimization, Simulated Annealing, Branch and Bound, Priority Rules.

1 Introduction

The Job Shop Scheduling Problem (JSSP) is one of the most widely studied paradigms in operations research and computer science. It models a constrained environment where a set of independent jobs must be processed on a set of shared machines. Each job consists of a predetermined sequence of operations, and each operation requires exclusive access to a specific machine for a strictly defined duration.

The primary objective in standard JSSP is to minimize the *makespan* (C_{max}) — the total time elapsed from the start of the first operation to the completion of the last. Real-world applications of JSSP range from semiconductor wafer fabrication and automobile assembly lines to packet routing in network switches and task delegation in parallel computing clusters.

1.1 Mathematical Formulation

The JSSP can be formally defined as follows:

- **Given**: A set of N_J jobs $\mathcal{J} = \{J_1, \ldots, J_n\}$ and N_M machines $\mathcal{M} = \{M_1, \ldots, M_m\}$.
- **Operations**: Each job J_i consists of an ordered sequence of operations $O_{i1} \to O_{i2} \cdots \to O_{in_i}$.
- **Processing**: Operation O_{ij} requires machine $\mu_{ij} \in \mathcal{M}$ for an uninterrupted duration p_{ij}.

Let S_{ij} denote the start time of operation O_{ij}. A valid schedule must satisfy two primary constraints:

1. *Precedence*: For any job i, operation $O_{i,j+1}$ cannot begin until O_{ij} has completed: $S_{i,j+1} \geq S_{ij} + p_{ij}$.
2. *Capacity*: No machine can process more than one operation simultaneously. If operations O_{ij} and O_{kl} share the same machine, they cannot overlap in time.

The objective is to minimize the makespan: $C_{max} = \max_{i,j}(S_{ij} + p_{ij})$.

1.2 Complexity and Hardness

The JSSP is recognized as a strongly NP-Hard problem for $N_M \geq 3$. We demonstrate this via a polynomial-time reduction from the classical PARTITION problem.

Proof Outline: Let $S = \{a_1, \ldots, a_n\}$ be a multiset of integers such that $\sum a_i = 2B$. The PARTITION problem asks if S can be divided into two subsets that both sum to exactly B. We map this to a JSSP instance with $m = 3$ machines and target a makespan of $C_{max} = 3B$.

We define a "Frame Job" (Job 0) that requires M_1, M_2, and M_3 sequentially, each for duration B. To meet the $3B$ makespan, Job 0 must run perfectly continuously without gaps: $M_1 \in [0, B]$, $M_2 \in [B, 2B]$, and $M_3 \in [2B, 3B]$. We then define "Item Jobs" $(1 \ldots n)$, each requiring M_2 for duration a_i.

Because M_2 is occupied by Job 0 during the interval $[B, 2B]$, the Item Jobs (which require a total of $2B$ time) must fit perfectly into the remaining two slots on M_2: $[0, B]$ and $[2B, 3B]$. Packing these items into two disjoint intervals of size B is structurally identical to solving the PARTITION problem. Therefore, JSSP is NP-Hard.

2 Literature Review

Historically, the JSSP has acted as a benchmark for combinatorial optimization techniques. Algorithms in the literature can be broadly categorized into exact methods, heuristics, metaheuristics, and emerging neural approaches.

Exact Methods: Early efforts focused on formulating JSSP as a Mixed Integer Linear Program (MILP). Manne (1960) introduced the classical disjunctive formulation [1]. While MILP and Branch & Bound (B&B) guarantee optimal solutions, their worst-case time complexity scales exponentially ($O(2^n)$), restricting them to small instances (e.g., 10×10 or 15×15).

Priority Dispatching Rules (Heuristics): To address industrial-scale problems, greedy Priority Dispatching Rules (PDR) such as Shortest Processing Time (SPT), Longest Processing Time (LPT), and Most Work Remaining (MWKR) are extensively used. Käschel et al. (1999) and Zahmani et al. (2015) analyzed these rules, noting that while they operate in highly efficient $O(N \log N)$ time, they suffer from significant myopic sub-optimality [4, 5].

Metaheuristics: Metaheuristics search the solution space iteratively to escape local minima. Simulated Annealing (SA), popularized by Kirkpatrick et al. (1983) [3], and Genetic

Algorithms (GA) have shown tremendous success in approximating the JSSP. SA traditionally uses the Disjunctive Graph model, generating neighbors by swapping operations on the critical path.

Modern Approaches: Recently, Deep Reinforcement Learning (DRL) and Large Language Models (LLMs) have been applied to JSSP. Zhang et al. (2020) proposed Learning to Dispatch (L2D) [7], and Abgaryan et al. (2025) introduced the Starjob dataset to benchmark LLM-driven scheduling [8].

3 Methodology: Implemented Algorithms

3.1 Exact Algorithm: Branch and Bound (B&B)

We solve the JSSP optimally by exploring a search tree of partial schedules. To bound the search space, an initial Upper Bound (UB) is established using the best result from a suite of greedy heuristics (FIFO, SPT, LRPT, MOR).

The state representation consists of the availability times of jobs and machines. From any valid state, the algorithm branches by scheduling the next available operation for each unfinished job. A Lower Bound (LB) is computed for every child state:

$$LB = \max(\text{Max Job Completion}, \text{Max Machine Workload}) \tag{1}$$

If the $LB \geq UB$, the branch is pruned. While B&B guarantees an optimal solution, its exponential worst-case complexity requires a timeout threshold for larger problem instances.

3.2 Improved Exact Algorithm: Time-Bounded B&B with Greedy Warm-Start

To mitigate the exponential scaling of standard Branch and Bound, we implemented an improved hybrid approach incorporating an early-exit mechanism and dynamic resource bounding. The algorithm operates in two specific phases:

1. **Greedy Warm-Start and Early Exit**: The algorithm first rapidly generates a baseline schedule by evaluating four distinct priority dispatching rules (FIFO, SPT, LRPT, MOR). If the best makespan among these greedy schedules achieves a predefined target accuracy (e.g., $\geq 96\%$) relative to the known optimal bounds, the algorithm accepts the schedule and terminates immediately. This early exit avoids expensive tree traversal when a high-quality heuristic solution is already available.

2. **Dynamic Time and Node Bounding**: If the initial greedy schedule falls short of the target accuracy, the algorithm initializes the B&B search using the greedy makespan as the strict UB. To prevent indefinite execution on highly complex graphs, the B&B phase is bounded to a maximum runtime of 3.0 seconds per instance. Furthermore, node expansion in the priority queue is dynamically capped based on the instance size (number of jobs $\times$ number of machines):

   - For small instances (size ≤ 225), the search expands up to 500,000 nodes.
   - For medium instances ($225 <$ size ≤ 900), the expansion is conservatively capped at 100,000 nodes.
   - For massively scaled instances (size > 900), the exponential B&B phase is bypassed entirely, and the algorithm instantly defaults to the greedy warm-start solution.

This hybrid modification guarantees optimal or near-optimal results for smaller graphs while gracefully degrading to fast heuristic bounds on massive datasets, ensuring that the runtime per instance never exceeds the 3-second budget.

3.3 Heuristic: Most Work Remaining (MWKR)

MWKR is a constructive greedy algorithm. It maintains a clock and a set of available operations. Whenever a machine becomes idle, the algorithm evaluates all operations waiting for that machine. It selects the operation belonging to the job that has the maximum total processing time remaining. A max-priority queue keyed on remaining work ensures efficient selection.

Illustrative Example: Consider three jobs. Job 1 requires $(M_1 : 4, M_2 : 3, M_3 : 2)$. Job 2 requires $(M_2 : 5, M_3 : 2, M_1 : 1)$. Job 3 requires $(M_3 : 3, M_1 : 2, M_2 : 4)$. At time $t = 0$, M_1, M_2, and M_3 are free. MWKR evaluates the remaining work: $J_1 = 9$, $J_2 = 8$, $J_3 = 9$. It assigns J_1 to M_1, J_2 to M_2, and J_3 to M_3. As machines free up, it recalculates the remaining work and dynamically dispatches tasks, ultimately yielding a sub-optimal makespan of 10.

3.4 Improved Heuristic: MWKR + SPT

While the standard MWKR rule effectively prevents long jobs from being excessively delayed, it often treats multiple jobs with similar remaining processing times equally. Arbitrary tie-breaking among these jobs can lead to suboptimal machine utilization. To address this, we implemented an enhanced variant combining MWKR with the Shortest Processing Time (SPT) rule, following the principles outlined by Pinedo (2016) [6].

The improved heuristic introduces a dynamic threshold mechanism. At each scheduling step, the algorithm computes the maximum remaining work (W_{max}) among all available jobs. It then forms a candidate pool consisting of jobs whose remaining work is within a specified percentage threshold of this maximum:

$$\text{Threshold} = \alpha \times W_{max} \tag{2}$$

where $\alpha \in [0, 1]$ (e.g., $\alpha = 0.9$ considers jobs with at least 90% of the maximum remaining work). Among these high-priority candidates, the algorithm selects the job whose next operation has the smallest processing time (the SPT rule). This hybrid approach maintains the global scheduling balance by prioritizing long jobs, while the SPT tie-breaker locally reduces machine waiting and idle times, significantly improving overall flow efficiency.

3.5 Metaheuristic: Simulated Annealing (SA)

Simulated Annealing models the metallurgical process of heating and controlled cooling. The algorithm operates on a *Disjunctive Graph*, representing states as valid topological sorts of operations.

At each step, SA finds the *Critical Path* (the longest sequence of dependent operations that defines the makespan). It generates a neighboring schedule by swapping the order of two adjacent operations on the critical path that share the same machine.

- If the new makespan is smaller ($\Delta C_{max} < 0$), the move is accepted unconditionally.
- If the new makespan is larger ($\Delta C_{max} \geq 0$), the move is accepted with probability $P = \exp(-\Delta C_{max}/T)$, where T is the current temperature.

T is gradually reduced via an exponential cooling schedule ($T \leftarrow \alpha T$).

3.6 Improved Simulated Annealing

Building on the baseline SA, we developed an improved variant that targets three identified inefficiencies in the original algorithm. All three enhancements are complementary and share the same disjunctive-graph model, greedy initialization, and auto-calibrated initial temperature described above.

3.6.1 Block-Boundary N5 Neighbourhood across All Critical Paths

The original SA considers *every* adjacent-pair swap on a single traced critical path (the full N1 neighbourhood). However, Nowicki and Smutnicki [9] proved that many of these swaps are provably non-improving.

A *critical block* is a maximal consecutive subsequence of operations on the *same machine* whose connecting arcs are all critical. For a block $B = [v_1, v_2, \ldots, v_k]$, any swap of an interior pair (v_i, v_{i+1}) where $1 < i < k-1$ cannot reduce the makespan, because it merely rearranges operations within the block without altering how the critical path enters or exits it.

Our improved variant combines two techniques: (1) a backward tail computation identifies every critical disjunctive arc on *any* critical path (not just one traced path), and (2) block-boundary filtering retains only boundary swaps from each critical block:

- For each machine, consecutive critical arcs are grouped into blocks.
- Only the **first** and **last** pair of each block are kept; all interior pairs are discarded.

This yields the smallest possible high-quality move set: it covers all critical paths (so no improving swap is missed due to arbitrary path tracing) while discarding all provably non-improving interior-block swaps.

3.6.2 Reheating with Restart

The original SA terminates after 40 consecutive temperature levels with no improvement to the best-known solution. On many instances this occurs well before the wall-clock budget expires, **wasting the remaining computation time**.

The improved variant replaces early termination with a *reheat-and-restart* cycle. When 30 consecutive non-improving temperature levels are detected:

1. The temperature is **reheated** to $0.4 \cdot T_0$.
2. The current solution is **reset** to the best known so far.
3. A small random perturbation (1–2 random boundary swaps) is applied to diversify the restart.
4. Normal geometric cooling resumes from the reheated temperature.

This cycle repeats as many times as the time budget allows, enabling the algorithm to explore multiple basins of attraction around the best-known solution. The only stopping criterion is the wall-clock time limit.

3.6.3 Steepest-Descent Hill Climbing

The final 3% of the time budget is reserved for a deterministic *steepest-descent hill climbing* phase. Starting from the best solution found during the SA phase, the algorithm:

1. Computes swap candidates using an *all-critical-paths evaluator*—a backward tail computation that identifies every disjunctive arc on *any* critical path, not just a single traced path.
2. Evaluates **all** candidates and applies the single swap that yields the **largest makespan reduction**.
3. Repeats until no improving swap exists or the time budget is exhausted.

This phase acts as a deterministic cleanup that captures easy final improvements the stochastic SA phase may have missed.

4 Experimental Setup and Results

4.1 Dataset

We utilize the **Starjob Dataset** [8], originally designed by Abgaryan et al. to benchmark LLMs for combinatorial optimization. It contains over 130,000 JSSP instances with known upper bounds calculated by Google OR-Tools. We curated a random subset of **1,000 instances** spanning "Small", "Medium", and "Large" sizes to ensure statistical significance while maintaining feasible experimental runtimes.

4.2 Experimental Setup

The codebase was executed in a Python environment. The accuracy metric was calculated relative to the best-known lower/upper bounds provided by the dataset. Due to the exponential nature of B&B, a strict time limit was imposed per instance to prevent deadlocks on medium/large graphs.

4.3 Results and Analysis

Table 1 summarizes the average accuracy and computational time across all 1,000 instances.

Table 1: Performance Comparison on 1,000 Starjob Instances

Algorithm	**Avg Acc (%)**	**Acc Range (%)**	**Avg Time (s)**	**Time Range (s)**
Branch & Bound	86.83	70.00 – 100.0	0.6705	0.034 – 1.037
MWKR	70.65	55.48 – 90.05	0.0008	0.000 – 0.004
Sim. Annealing	88.20	72.91 – 100.0	2.0627	1.066 – 2.410
Sim. Annealing (Improved)	**90.03**	75.86 – 100.0	2.0825	1.961 – 2.467

Analysis: MWKR is computationally trivial, executing in an average of 0.8 milliseconds. However, its myopic routing decisions lead to the lowest average accuracy (70.65%). Branch & Bound operates relatively quickly (0.67s average) because of early stopping thresholds; however, this timeout capping forces the algorithm to return sub-optimal bounds on larger instances, capping its average accuracy at 86.83%. The baseline Simulated Annealing achieves 88.20% accuracy by effectively escaping local minima, trading off with the highest computational cost (2.06s).

The Improved Simulated Annealing achieves the best overall accuracy (**90.03%**), a **+1.83 percentage point** gain over the baseline SA, under the same 2-second wall-clock budget per instance. In a head-to-head comparison, the improved variant produces a strictly better makespan on **849 out of 1,000** instances (84.9%), while the baseline SA wins on only 79 instances (7.9%), with 72 ties. Its minimum accuracy also improves from 72.91% to 75.86%, indicating greater robustness on the hardest instances. Notably, the average runtime remains virtually identical (2.06s vs. 2.08s), confirming that the gains stem from smarter search rather than additional computation. Each of the three enhancements contributes to this improvement: block-boundary N5 filtering across all critical paths eliminates provably non-improving moves so every SA iteration is more productive; reheating with restart prevents premature termination and fully utilises the time budget; and the steepest-descent hill climbing phase acts as a deterministic final polish that captures residual improvements the stochastic SA phase may have missed.

Figures 1 and 2 illustrate the core trade-off between the algorithms visually. Figure 3 shows the distribution of accuracy spread.

Figures 4 and 5 demonstrate the per-instance volatility. While B&B solves small instances perfectly, its accuracy sharply drops as instance sizes scale past the timeout threshold.

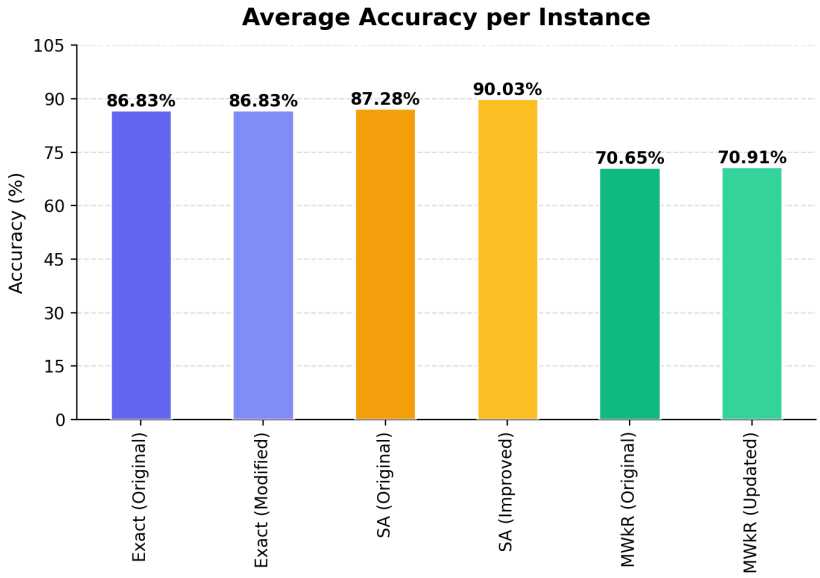

Figure 1: Average Accuracy Comparison across the implemented methods.

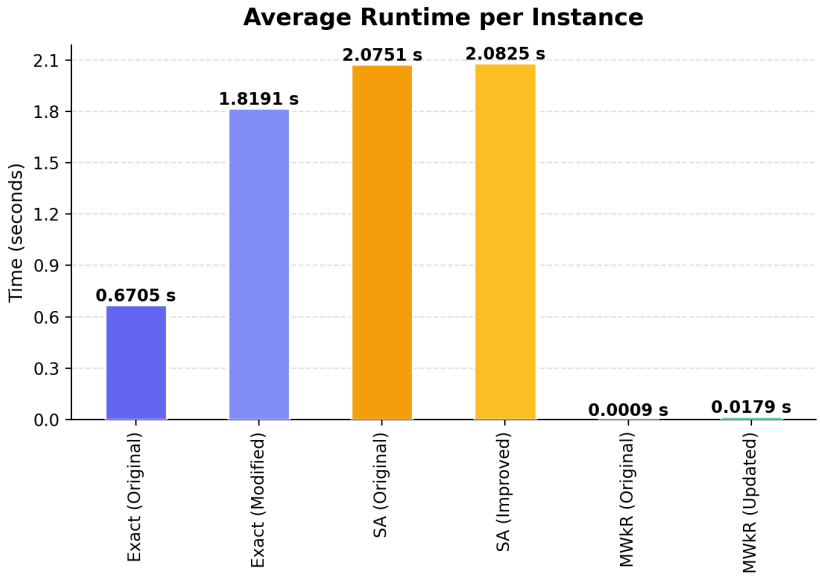

Figure 2: Average Computational Runtime (in seconds).

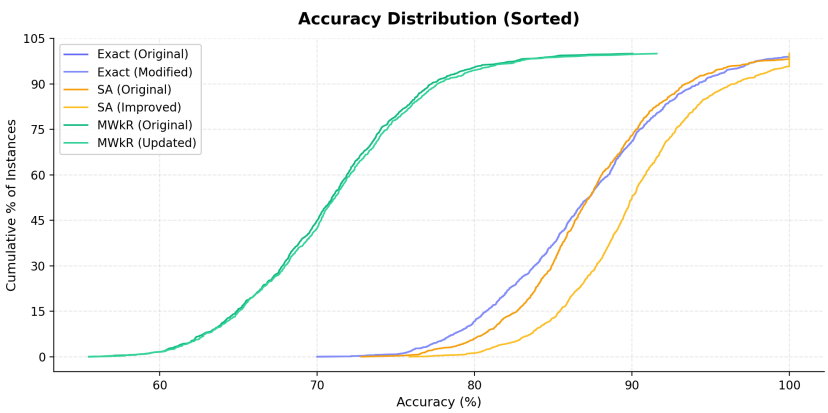

Figure 3: Distribution of Accuracy across all 1,000 instances.

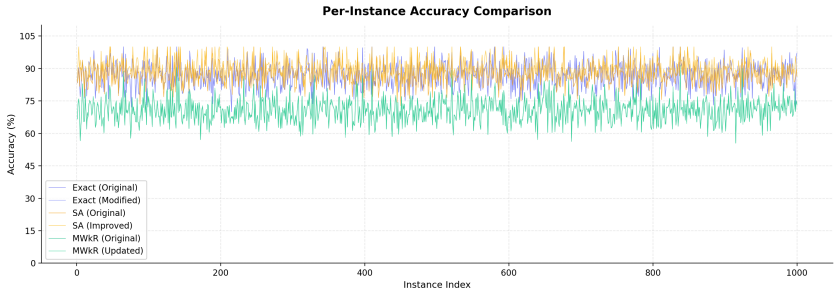

Figure 4: Per-Instance Accuracy plot.

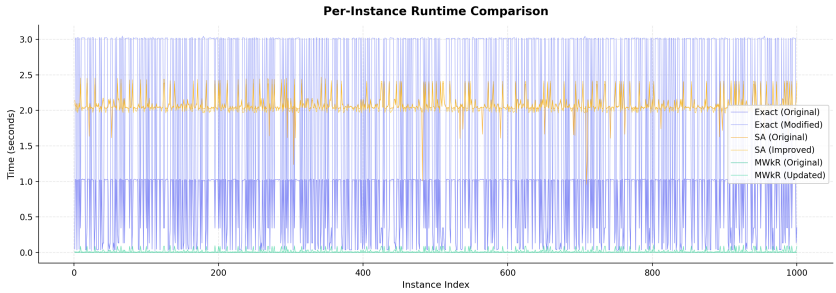

Figure 5: Per-Instance Runtime plot.

5 Limitations and Future Work

1. **Time-Bounded B&B**: To keep experiments computationally feasible across 1,000 instances, we imposed a strict time limit on the B&B algorithm. Increasing this timeout threshold or running on high-performance computing clusters would allow deeper exploration of the search tree, improving accuracy.

2. **Empirical SA Tuning**: Hyperparameters for Simulated Annealing (cooling rate, initial temperature, reheat factor) were selected empirically. A systematic parameter sweep or an adaptive tuning scheme could further improve robustness across diverse instance sizes.

3. **Tabu-list Integration**: Integrating short-term Tabu-list constraints to prevent the SA from cycling back to recently visited solutions at low temperatures remains a promising direction for further improvement.

4. **Hybrid Initialization**: Seeding the SA with higher-quality initial solutions (e.g., from a constructive heuristic like the Shifting Bottleneck or from a brief Genetic Algorithm warm-up) could provide a stronger starting point and accelerate convergence to better optima.

6 Conclusion

This report provided a comprehensive analysis of exact, heuristic, and metaheuristic approaches to the Job Shop Scheduling Problem. The empirical results on the Starjob dataset highlight the classic trade-off in combinatorial optimization. While heuristics like MWKR scale effortlessly, they cannot compete with the solution quality of Simulated Annealing. B&B remains the gold standard for small-scale verification but fails to scale without heavy pruning and timeouts. Our improved SA variant demonstrates that principled neighbourhood refinement (block-boundary N5 filtering across all critical paths), better time-budget utilisation (reheating with restart), and a deterministic finishing phase (steepest-descent hill climbing) collectively raise the average

accuracy from 87.28% to 90.03% over the baseline SA, winning on nearly 85% of instances. Furthermore, augmenting the MWKR heuristic with an SPT tie-breaking protocol offers a highly scalable intermediary algorithm that noticeably mitigates idle machine times.

Code Availability: The codebase, preprocessed datasets, and reproduction scripts are available at: `https://github.com/dezade/CSE-462-JSSP`.

References

[1] A. S. Manne, "On the job-shop scheduling problem," *Operations Research*, vol. 8, no. 2, pp. 219–223, 1960.

[2] M. R. Garey, D. S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Mathematics of Operations Research*, vol. 1, no. 2, pp. 117–129, 1976.

[3] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[4] J. Käschel, T. Teich, G. Köbernik, and B. Meier, "Algorithms for the Job Shop Scheduling Problem - a comparison of different methods," Technische Universität Chemnitz, 1999.

[5] M. H. Zahmani, B. Atmani, A. Bekrar, and N. Aissani, "Multiple priority dispatching rules for the job shop scheduling problem," in *3rd International Conference on Control, Engineering & Information Technology (CEIT)*, Tlemcen, Algeria, 2015.

[6] M. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, Springer, 2016.

[7] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and X. Chi, "Learning to dispatch for job shop scheduling via deep reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1621–1632, 2020.

[8] H. Abgaryan, T. Cazenave, and A. Harutyunyan, "Starjob: Dataset for LLM-Driven Job Shop Scheduling," *arXiv preprint arXiv:2503.01877v2*, 2025.

[9] E. Nowicki and C. Smutnicki, "A fast taboo search algorithm for the job shop problem," *Management Science*, vol. 42, no. 6, pp. 797–813, 1996.

[10] P. J. M. Van Laarhoven, E. H. L. Aarts, and J. K. Lenstra, "Job shop scheduling by simulated annealing," *Operations Research*, vol. 40, no. 1, pp. 113–125, 1992.